System Architecture Template

Agile Software Engineering Capstone

Document field	Team entry
Project title	Student Savings
Team name	Ninja Penguins
Team members	Lupe Campos Joey Contreras Doga Eski Jonathan Ochoa
Student document owner	Doga Eski
Instructor	Shah, Syed Jawad Hussain
Sponsor or client	Commerce Bank
Version and date	0.5, November 9th, 2026
Approval status	In review

Note: Consider this document as template only and feel free to customize if needed.

How to use this template

Use this document to explain the system's important structures, how its parts work together, and why the team made significant technical decisions. A teammate should be able to follow a user request through the system, find where data lives, and understand the main quality and security choices.

Replace each bracketed prompt with your team's information. Remove unused example rows and all remaining prompts before submission. Keep the document current as the project changes.

Create a thin first version during Sprint 0 or Sprint 1. Update it whenever the team changes a major component, interface, data model, deployment approach, security boundary, or important technical decision.

Revision history

Version	Date	Author	Summary of change	Reviewer or approval
0.1	9-25-2025	Lupe Campos	Initial draft	Lupe Campos
0.2	10-2-2026	Lupe Campos	Updated purpose and scope and architecture drivers.	Lupe Campos
0.3	10-9-2026	Lupe Campos	Updated Key use cases and	Lupe Campos
0.4	10-16-2026	Lupe Campos	Updated 2.1-5.1	Lupe Campos
0.5	11-6-2026	Lupe Campos	Updated 5.2-7.1	Lupe Campos

Document outline

Section	Purpose
1 Architecture overview	Purpose, scope, status, and a short explanation of the system
2 Architecture drivers	Use cases, quality attributes, constraints, assumptions, and risks
3 Stakeholders and concerns	What different readers need from the architecture
4 Architecture at a glance	Selected architecture style, major elements, and rationale
5 System context view	Users, external systems, system boundary, and interactions
6 Logical view	Containers, components, responsibilities, and dependencies
7 Process and runtime view	Requests, processes, concurrency, state, and failure behavior
8 Development view	Source organization, modules, ownership, and build structure
9 Deployment view	Where software runs and how environments connect
10 Use case view	Scenarios that validate the other architecture views
11 Data architecture	Entities, relationships, ownership, validation, and migrations
12 Interfaces and APIs	Contracts, authentication, validation, errors, and versioning
13 Security and privacy	Trust boundaries, threats, secrets, identity, and data rules
14 Quality attributes	Performance, reliability, accessibility, maintainability, and testability
15 Architecture decisions	Short records of major decisions and consequences
16 Traceability and risks	Links to requirements, tests, prototypes, and unresolved concerns

Section	Purpose
17 Architecture evolution	How the team reviews and updates the architecture
18 Review and submission	Approvals and final completeness check

1 Architecture overview

1.1 Purpose and scope

This document describes the system architecture for student savings, a web-based personal finance management application. It covers the frontend web application, backend API, database, user authentication, and the integration between these components.

The system allows users to register and log in, manage financial accounts, track income and expenses, create monthly budgets, monitor loans, establish savings goals, and view their financial information through a centralized dashboard.

The architecture is intended for developers, project team members, and other stakeholders involved in developing, maintaining, and evaluating the application.

Example: This document describes the web application, API, database, automated test pipeline, and demo deployment for Budget Buddy. It does not describe the internal design of third-party libraries or any real banking integration.

1.2 Architecture status

Item	Team entry
Architecture version	Version 0.5 – November 6, 2026
Current status	Partly implemented
Implementation baseline	Current development version submitted to the team's GitHub repository
Known gaps	CSV transaction importing is a potential future feature. Remaining authentication, profile, settings, and dashboard functionality may require additional implementation and testing.

1.3 Short architecture narrative

Student savings uses a web-based frontend, a backend REST API, and a database to provide personal finance management functionality. Users interact with the frontend to register, log in, and manage their financial information. The frontend communicates with the backend API, which handles authentication, validates requests, processes business logic, and retrieves or updates information in the database. The backend manages user accounts, transactions, budgets, loans, and savings

goals. A centralized dashboard endpoint gathers the user's financial information and returns it in a format that the frontend can display. All financial records are associated with individual users to keep their information separate. The application is developed and maintained through a shared GitHub repository.

---

2 Architecture drivers

Architecture drivers are requirements or risks that strongly influence the system structure. Focus on the few items that change architectural choices.

2.1 Key use cases

Use case ID	Use case	Why it affects architecture	Related view or decision
UC-AUTH-02	Create an account	Requires user registration, secure password handling, and storing user records in the database.	Security and data views
UC-AUTH-03	Sign in	Requires credential verification and authentication to protect user-specific resources.	Security and runtime views
UC-ACC-01	Add a financial account	Requires storing user-owned financial accounts, enforcing unique account names per user, and supporting checking, savings, and credit card types.	Logical and data views
UC-ACC-02	Record a transaction	Requires linking transactions to user accounts, storing transaction details, and calculating account balances using the opening balance and associated transactions.	Logical, data, and runtime views
UC-BUD-01	Create a monthly budget	Requires storing monthly spending limits by category and calculating budget usage from matching expense transactions.	Logical and data views
UC-LOAN-02	Track loan payments	Requires storing loan balances, interest rates, monthly payments, and upcoming due dates.	Logical and data views
UC-GOAL-01	Create a financial goal	Requires storing individual savings goals, including saved amounts, target amounts, and target dates.	Logical and data views

Use case ID	Use case	Why it affects architecture	Related view or decision
UC-UC-SPEND-01	Review spending	Requires collecting and summarizing transaction information by category and displaying financial summaries through the dashboard.	Logical and runtime views

2.2 Quality attributes

Quality ID	Quality scenario	Measure or response	Architecture effect
QA-01	When a user signs in, the system must protect their financial information from access by other users.	Verify that authenticated users can only access their own financial records.	Spring Security, authentication, authorization, and user-specific database queries.
QA-02	When a user opens the dashboard, the system must retrieve and display their financial information efficiently.	Dashboard data is returned through one API request and displays the correct information.	Centralized dashboard endpoint that gathers accounts, transactions, budgets, loans, and goals.
QA-03	When a user adds a transaction, the account balance must reflect the transaction amount.	Account balance should equal the opening balance plus all associated transactions.	Backend balance calculations and transaction-to-account relationships.
QA-04	When a user reviews a monthly budget, the system must calculate spending using the correct month's transactions and category.	Budget spending totals should match the sum of applicable negative transactions.	Backend budget calculation logic using transaction dates, categories, and user ownership.
QA-05	When a user creates a financial account, the system must prevent duplicate account names for that user.	Duplicate account names are rejected while different users can use the same name.	Database constraints or backend validation based on user ownership.
QA-06	When a user creates, updates, or deletes financial records, the changes must be saved correctly.	Verify that CRUD operations correctly create, retrieve, update, and delete accounts, transactions, budgets, loans, and goals.	REST API endpoints, Spring Data JPA, and PostgreSQL persistence.
QA-07	When a user reviews their loans, the system must display accurate remaining balances and identify the next upcoming payment.	Loan totals and next payment dates match the stored loan records.	Backend loan calculations and due-date comparison logic.
QA-08	When a user creates a savings goal, the system must track progress toward the target amount and date.	Saved amounts, target amounts, and target dates are correctly stored and displayed.	Goal data model and user-specific database operations.

2.3 Constraints assumptions and architecture risks

Type	Statement	Source	Architecture effect	Owner or validation
Constraint	Use only fabricated financial and card data.	Project requirements	No real banking connections or payment processing are included in the project scope.	Whole team
Constraint	Users must have individual accounts and only access their own financial information.	Project requirements	Backend must enforce authentication and restrict access to user-owned data.	Joey Contreras(Backend) validates authorization.
Constraint	The application must support checking, savings, and credit card accounts.	Project requirements	Account functionality must support the required account types.	Backend team validates account functionality.
Assumption	The development and demo environments support the selected frontend, backend, and database technologies.	GitHub repository and development environment	Deployment uses React, Spring Boot, PostgreSQL 16, and Docker Compose.	Development team verifies deployment compatibility.
Assumption	Users will manually enter their financial accounts, transactions, loans, and savings goals.	Project scope	Backend must support manual data entry and CRUD operations.	The whole team validates through testing.
Assumption	The dashboard is intended to retrieve financial information through a centralized dashboard endpoint.	Team architecture design	Frontend depends on the backend returning the required financial information in the expected structure.	Joey Contreras and Doga Eski(Frontend) validate integration.
Risk	Incorrect transaction amounts or calculations could cause inaccurate account balances.	Transaction and account functionality	Backend calculations must correctly combine opening balances with transactions.	Backend team validates with calculation tests.
Risk	Users could access another user's financial records if authorization is not properly enforced.	Security requirements	Backend queries and API operations must restrict access to the authenticated user's records.	Backend team performs authorization..
Risk	Dashboard performance could decrease as the amount of financial data increases.	Dashboard requirements	Database queries and dashboard responses may require optimization.	Backend team performs performance testing.
Risk	CSV transaction importing may not be completed within the project timeline.	Project schedule and future feature plans	CSV importing may need to be excluded from the initial release.	Team evaluates available time before Sprint 4.

3 Stakeholders and concerns

Stakeholder	Main concerns	Architecture information needed	View or evidence
Users	Usability, privacy, correct behavior	User journeys and visible system behavior	Use case and logical views
Developers	Clear responsibilities and interfaces	Components, APIs, dependencies, and source structure	Logical and development views
Testers	Testable boundaries and failure behavior	Interfaces, quality scenarios, environments, and logs	Runtime, deployment, and quality sections
Sponsor or instructor	Scope, feasibility, risk, and evidence	Context, major decisions, tradeoffs, and status	Overview, drivers, ADRs, and traceability

4 Architecture at a glance

4.1 Selected architecture style

Student Savings uses a combination of Client-Server and Layered (3-Tier) Architecture, consisting of a Presentation Layer, Application Layer, and Data Layer. The Presentation Layer uses React to provide the web interface, allowing users to manage their financial accounts, transactions, budgets, loans, and savings goals. The Application Layer uses a Spring Boot backend REST API to handle requests, authentication, business logic, financial calculations, and communication with the database. The Data Layer uses PostgreSQL 16 to store user information and financial records.

The application uses Docker Compose to organize and run the frontend, backend, and database as separate services. The frontend communicates with the backend through REST API requests, while the backend communicates with PostgreSQL to store and retrieve information.

This architecture separates the frontend, backend, and database, making the system easier to develop, maintain, test, and expand. It also allows team members to work on different components independently while maintaining organized access to financial data.

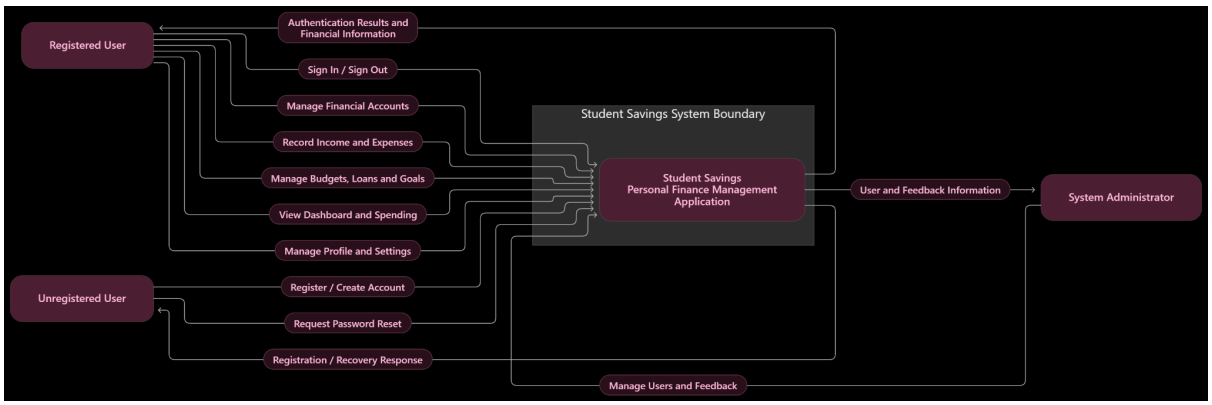
4.2 Major architecture elements

Element	Responsibility	Important interface	Data owned or used	Team owner
Web client	Uses React to display application pages, handle user interactions, and send requests to the backend.	REST API requests using HTTP and JSON.	Temporary UI state and financial information displayed on the dashboard.	Doga Eski

Element	Responsibility	Important interface	Data owned or used	Team owner
API	Uses Spring Boot to handle authentication, validate requests, apply business rules, perform financial calculations, and return responses.	REST endpoints using HTTP and JSON.	User information, accounts, transactions, budgets, loans, and savings goals through backend services and repositories.	Joey Contreras
Database	Uses PostgreSQL 16 to store, retrieve, and maintain persistent application data.	SQL connections through Spring Data JPA and the PostgreSQL JDBC driver.	User records, financial accounts, transactions, budgets, loans, and savings goals.	Joey Contreras
Deployment Environment	Uses Docker Compose to run and connect the frontend, backend, and database as separate services.	Docker Compose service configuration and internal network connections.	Application configuration, environment variables, and database connection settings.	Whole team

5 System context view

5.1 Context diagram



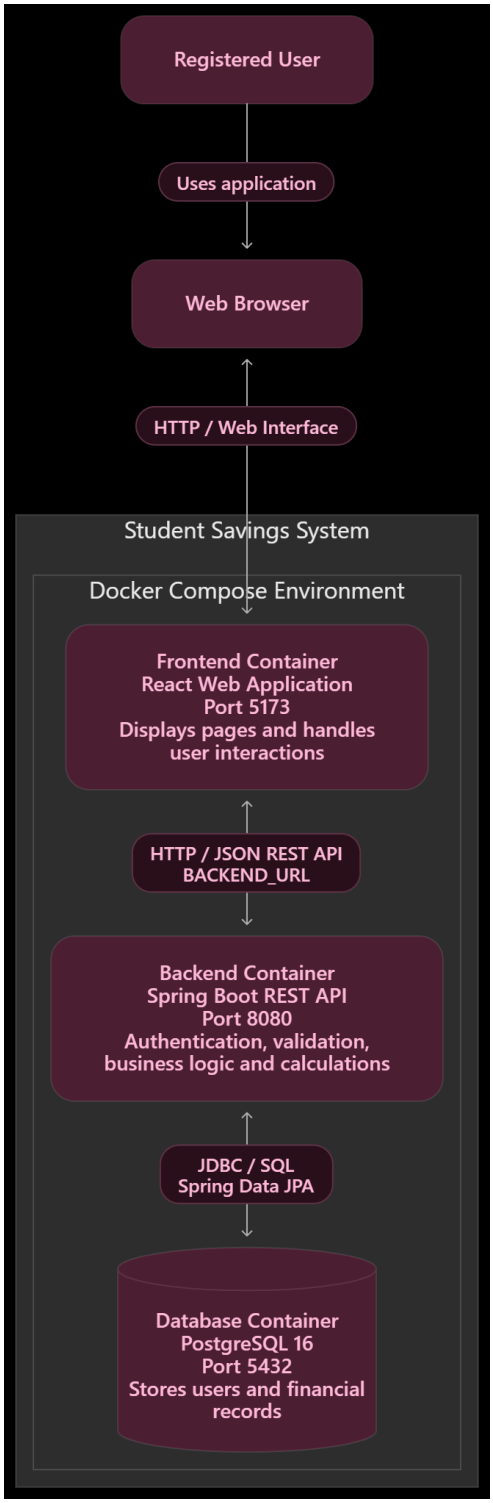
5.2 Context explanation

Actor or external system	Interaction with the system	Information exchanged	Trust or dependency concern
Registered user	Uses the application to manage financial information.	Login credentials, profile, accounts, transactions, budgets, loans, and savings goals.	Users should only access their own financial data.

Actor or external system	Interaction with the system	Information exchanged	Trust or dependency concern
Unregistered user	Registers for an account or requests a password reset.	Registration information and account recovery information.	User information must be validated and protected.
System administrator	Manages users and reviews feedback.	User information, account status, and feedback submissions.	Administrative access must be restricted to authorized users.

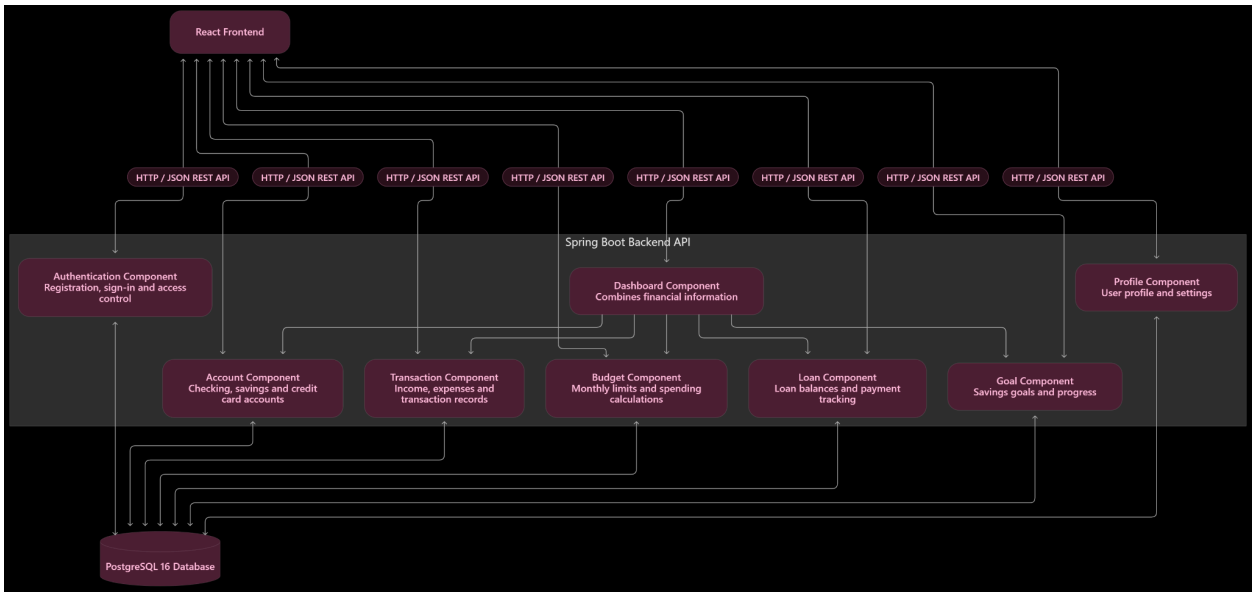
6 Logical view

6.1 Container diagram



6.2 Component view

6.2 Component View

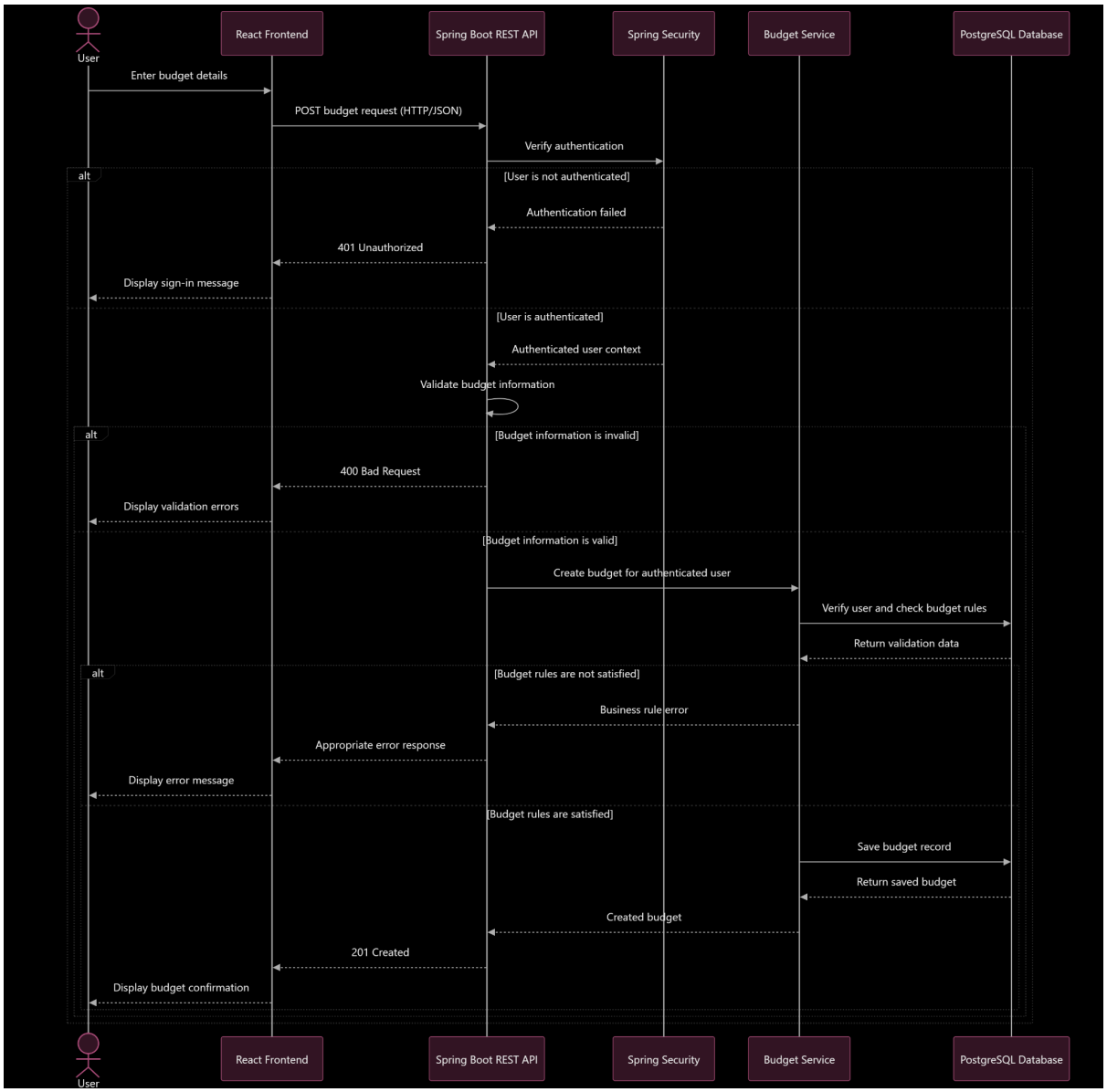


Component	Responsibility	Depends on	Provided interface	Requirement or use case
Authentication component	Handles registration, login, logout, and password recovery.	Spring Security and user data persistence.	Authentication API endpoints	UC-AUTH-01, UC-AUTH-02, UC-AUTH-03, UC-AUTH-04
Account component	Creates, retrieves, updates, and deletes checking, savings, and credit card accounts.	Database persistence and authenticated user information.	Account API endpoints	UC-ACC-01
Transaction component	Records income and expenses and associates transactions with financial accounts.	Database persistence and account information.	Transaction API endpoints	UC-ACC-02
Budget component	Manages monthly spending limits by category and calculates budget usage.	Budget and transaction data.	Budget API endpoints	UC-BUD-01

Component	Responsibility	Depends on	Provided interface	Requirement or use case
Loan component	Stores loan information and tracks remaining balances, monthly payments, and upcoming due dates.	Database persistence and loan information.	Loan API endpoints	UC-LOAN-01, UC-LOAN-02
Goal component	Creates and updates savings goals and tracks progress toward target amounts and dates.	Database persistence and user information.	Goal API endpoints	UC-GOAL-01, UC-GOAL-02
Dashboard component	Collects financial information and prepares a combined dashboard response.	Account, transaction, budget, loan, and goal components.	Dashboard API endpoint	UC-SPEND-01
Profile component	Manages user profile information and settings.	User data persistence and authenticated user information.	Profile API endpoints	UC-PROF-01

7 Process and runtime view

7.1 Key request or event flows



7.2 Processes concurrency state and failure

Concern	Design decision	Failure behavior	How the team will test or observe it
Session state	Use Spring Security to manage authentication and protect user-specific resources. The exact session or token mechanism must match the implementation.	Invalid or expired authentication should result in an appropriate authentication error, such as 401 Unauthorized.	Test login, logout, expired authentication, and protected API access.
Concurrent update	Use database transactions where needed to protect financial records during simultaneous operations.	Conflicting updates should be handled without silently losing or overwriting data.	Test simultaneous account and transaction updates.
Database failure	Backend communicates with PostgreSQL through Spring Data JPA and handles database errors.	API should return an appropriate server error without exposing sensitive database information.	Test database connection failures and review application logs.
Invalid requests	Backend validates incoming requests before processing them.	Invalid information should return an appropriate validation response, such as 400 Bad Request.	Test missing fields, invalid amounts, and incorrect input formats.
Unauthorized access	Spring Security and backend authorization checks should restrict access to records belonging to the authenticated user.	Unauthenticated requests should return 401; authenticated users without permission should receive 403 or an appropriate access error.	Test access using multiple user accounts.
Dashboard failure	Dashboard component should retrieve the user's financial information through the backend components.	API should return an appropriate error if dashboard information cannot be retrieved.	Test dashboard retrieval and integration with accounts, transactions, budgets, loans, and goals.

8 Development view

8.1 Source and module organization

Repository or module	Purpose	Main dependencies	Build or test command	Owner
Frontend	Frontend application, pages, and user interface.	Backend REST API and UI components	Depends on frontend framework	Doga Eski
Backend	Spring Boot API, controllers, services, repositories, and authentication.	Spring Boot, database, and security libraries	Not established yet	Joey Contreras

Repository or module	Purpose	Main dependencies	Build or test command	Owner
Account module	Creates, retrieves, updates, and deletes checking, savings, and credit card accounts.	Backend API, database, and authenticated user information	Not established yet	Joey Contreras
Transaction module	Routes, services, repositories, and authentication	Database and session library	Not established yet	Joey Contreras
Budget module	Creates monthly spending limits by category and calculates spending from transactions.	Transaction module and database	Not established yet	
Loan module	Stores loan information and tracks remaining balances, monthly payments, and upcoming due dates.	Backend API and database	Not established yet	Joey Contreras
Goal module	Creates and updates savings goals, including saved amounts, target amounts, and target dates.	Backend API and database	Not established yet	Joey Contreras
Dashboard module	Gathers account, transaction, budget, loan, and goal information into one response for the frontend.	Account, transaction, budget, loan, and goal modules	Not established yet	Joey Contreras

8.2 Team and integration structure

What to write: *The Ninja Penguins team is organized into four main roles: backend development, frontend development, documentation, and project management. The team uses one shared GitHub repository containing separate `backend` and `frontend` folders. This organization allows frontend and backend development to happen separately while keeping both parts of the application within the same project.*

Doga Eski: Front End Developer

Develops the frontend application, user interface, pages, and communication with the backend API.

Joey Contreras: Backend Developer

Develops the Spring Boot backend, REST API endpoints, business logic, and database interactions.

Lupe Campos: Documentation

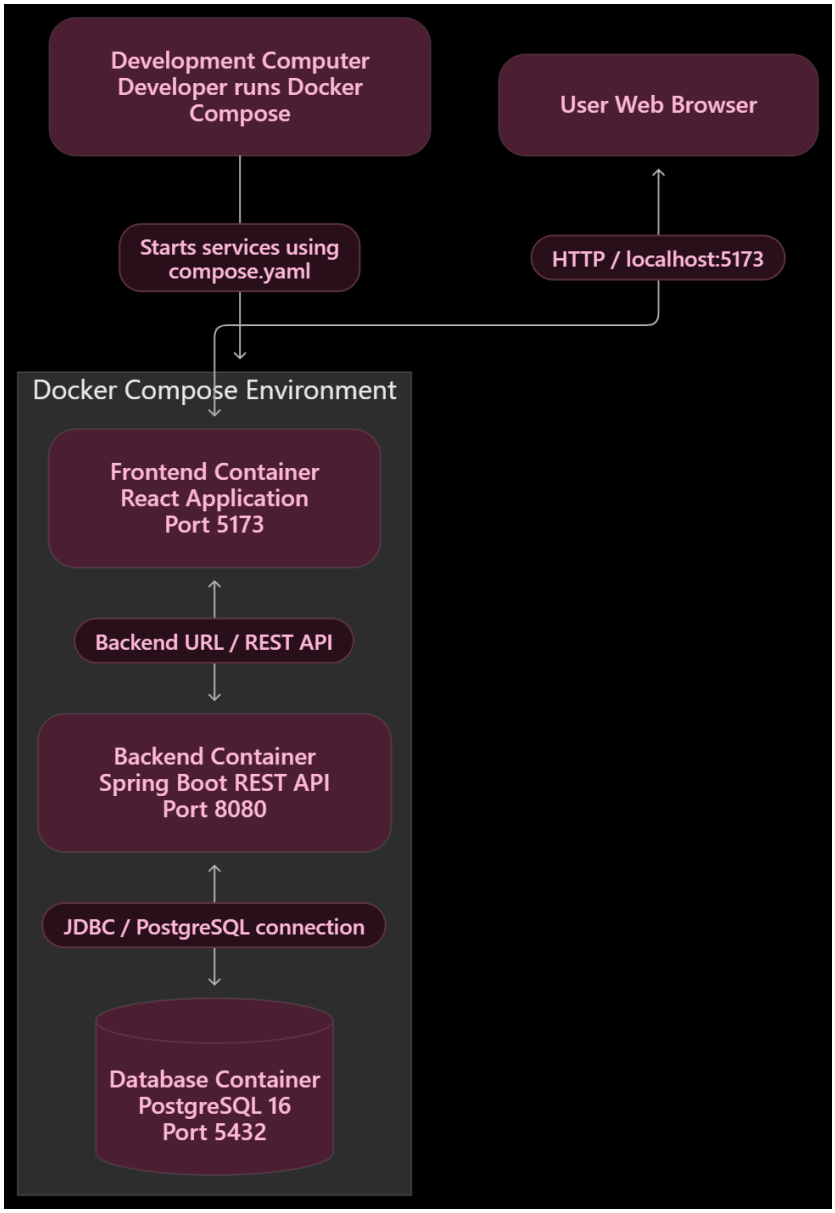
Maintains project documentation, architecture descriptions, requirements, and development records.

Jonathan Ochoa: Project Manager

Coordinates team activities, tracks progress, organizes tasks, and helps ensure project milestones are completed.

9 Deployment view

9.1 Deployment diagram



9.2 Environment inventory

Environment	Purpose	Deployed elements	Data	Configuration and secrets	Owner
Local development	Develop, run, and debug the application.	React frontend, Spring Boot backend, PostgreSQL database using Docker Compose.	Fabricated developer data.	Local environment variables and Docker Compose configuration. Secrets should not be committed to GitHub.	Each developer
Test or demo	Test frontend and backend integration and demonstrate application functionality.	React frontend, Spring Boot API, and PostgreSQL database.	Fabricated test/demo data.	Environment-specific configuration and database credentials stored outside source control.	Joey Contreras and Doga Eski
Production	Host the completed application for end users.	React frontend, Spring Boot backend, and PostgreSQL database.	Fabricated user financial data, as required by project scope.	Secure production environment variables and protected database credentials.	Jonathan Ochoa

10 Use case view

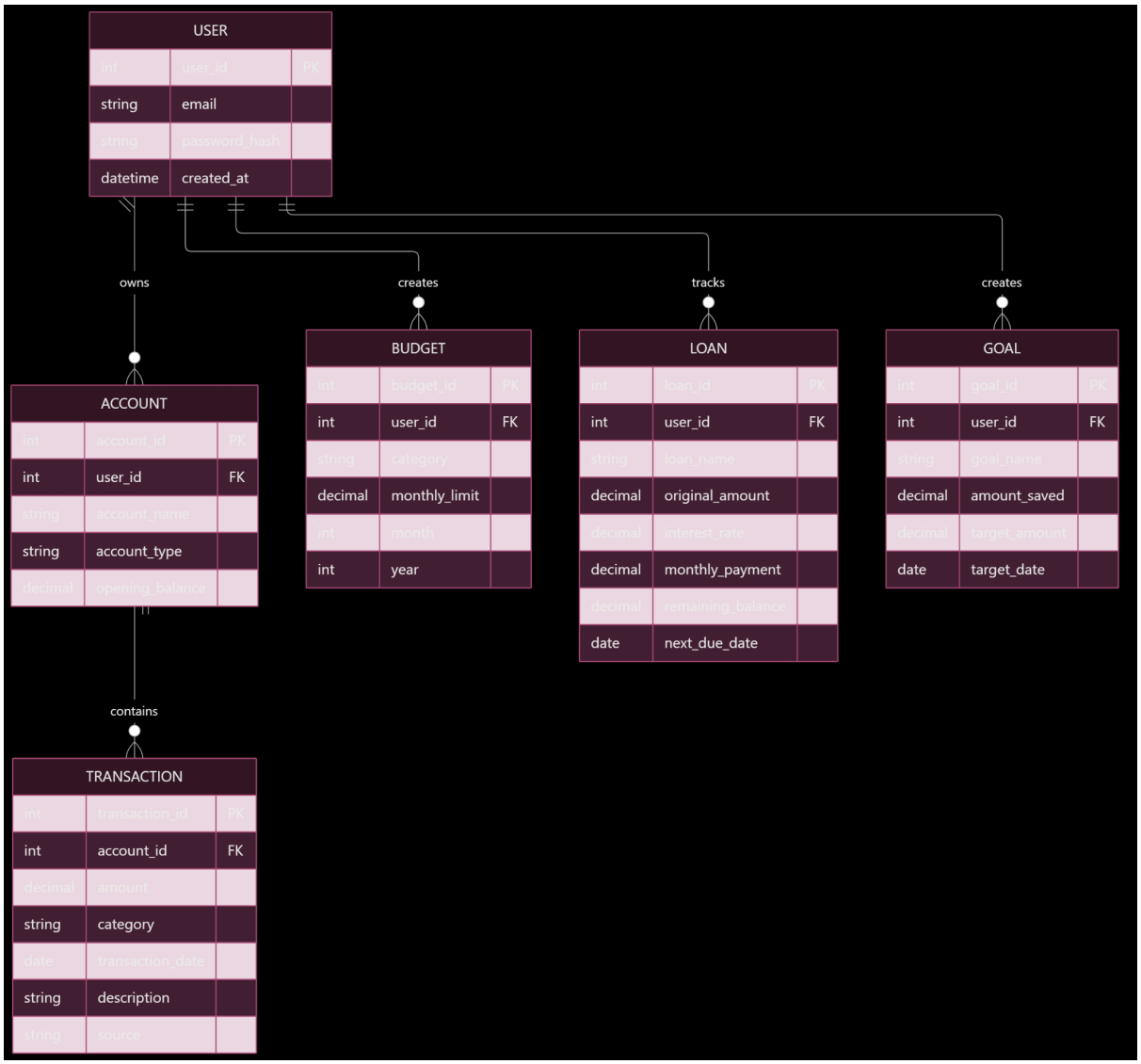
The use case view is the plus one view. It shows how the logical, runtime, development, and deployment views work together for an important scenario.

Scenario	Logical elements	Runtime flow	Deployment nodes	Evidence
Register and sign in	Frontend forms, authentication controller, authentication service, user repository	User enters credentials → frontend sends request → backend validates information → user information is stored or verified → authentication result is returned.	Browser, React frontend container, Spring Boot backend container, PostgreSQL database container	Registration and login functionality; manual testing or demonstration.
Record a transaction	Transaction page, transaction controller, transaction service, account module, database	User enters transaction details → frontend sends request → backend validates the information and account ownership → transaction is saved → updated financial information is returned.	Browser, React frontend container, Spring Boot backend container, PostgreSQL database container	Transaction functionality and manual testing or demonstration.

Scenario	Logical elements	Runtime flow	Deployment nodes	Evidence
View financial dashboard	Dashboard page, dashboard controller, dashboard service, account, transaction, budget, loan, and goal modules	User opens dashboard → frontend requests dashboard information → backend gathers the user's financial records → backend returns dashboard data → frontend displays the information.	Browser, React frontend container, Spring Boot backend container, PostgreSQL database container	Dashboard implementation and frontend/backend integration demonstration.

11 Data architecture

11.1 Data model



Relationships:

User → Account

One user can have multiple financial accounts.

Account → Transaction

One account can contain multiple transactions.

User → Budget

One user can create multiple monthly budgets.

User → Loan

One user can track multiple loans.

User → Goal

One user can create multiple savings goals.

11.2 Data dictionary and ownership

Entity or data set	Purpose	Important fields	Owner or access rule	Validation retention or migration
User	Stores income and expense records associated with financial accounts.	User ID, email, password hash, created date	API only; users can access their own profiles.	Unique email; passwords must never be sAPI only; users can access their own profiles.tored in plain text.
Account	Stores a user's monthly spending plan	Account ID, user ID, account name, account type, opening balance	Each account belongs to one user; users can only manage their own accounts.	Account names must be unique per user; account type must be valid.
Transaction	Stores income and expense records associated with financial accounts.	Transaction ID, account ID, amount, category, date, description, source	Transactions belong to an account owned by the authenticated user.	Amounts must be valid; negative amounts represent expenses and positive amounts represent income.

Entity or data set	Purpose	Important fields	Owner or access rule	Validation retention or migration
Budget	Stores a user's monthly spending limits by category.	Budget ID, user ID, month, year, category, monthly limit	Queries must use authenticated user ID	Monthly limits must be positive; spending is calculated using matching transactions.
Loan	Stores loan information and tracks repayment progress.	Loan ID, user ID, loan name, original amount, interest rate, monthly payment, remaining balance, next due date	Each loan belongs to one user; users can only access their own loans.	Loan amounts and payments must be valid; remaining balance should not be negative.
Goal	Stores a user's savings goals and progress.	Goal ID, user ID, goal name, amount saved, target amount, target date	Each goal belongs to one user; users can only access their own goals.	Target amount must be positive; saved amount cannot be negative.

12 Interfaces and APIs

12.1 Interface catalog

Interface ID	Provider and consumer	Purpose	Protocol or format	Authentication	Version or status
API-01	Frontend to Backend	Register a new user	POST `/api/auth/register` with JSON	Not required; new user	Existing endpoint to verify
API-02	Frontend to Backend	Sign in a registered user	POST `/api/auth/login` with JSON	Credentials required	Existing endpoint to verify
API-03	Frontend to Backend	Retrieve user accounts	GET `/api/accounts` with JSON response	Required	Planned/verify
API-04	Frontend to Backend	Create a financial account	POST `/api/accounts` with JSON	Required	Planned/verify

Interface ID	Provider and consumer	Purpose	Protocol or format	Authentication	Version or status
API-05	Frontend to Backend	Manage financial transactions	GET, POST, PUT, DELETE `/api/transactions` with JSON	Required	Planned/verify
API-06	Frontend to Backend	Create and manage monthly budgets	GET, POST, PUT, DELETE `/api/budgets` with JSON	Required	Planned/verify
API-07	Frontend to Backend	Create and manage loans	GET, POST, PUT, DELETE `/api/loans` with JSON	Required	Planned/verify
API-08	Frontend to Backend	Create and manage savings goals	GET, POST, PUT, DELETE `/api/goals` with JSON	Required	Planned/verify
API-09	Frontend to Backend	Retrieve dashboard information	GET `/api/dashboard` with JSON response	Required	Planned/verify
API-10	Frontend to Backend	Retrieve and update user profile	GET, PUT `/api/profile` with JSON	Required	
API-11	Frontend to Backend	Sign out the current user	POST `/api/auth/logout` with JSON	Required	

12.2 Contract details

Contract field	Team entry
Endpoint or interface	`POST /api/transactions`
Purpose	Allows an authenticated user to record an income or expense transaction associated with one of their financial accounts.
Request example	`{"accountId": 1, "amount": -45.50, "category": "Groceries", "date": "2026-11-09", "description": "Food shopping", "source": "manual"}`
Success response	`201 Created` – Returns the newly created transaction, including its transaction ID and submitted information.
Validation errors	`400 Bad Request` – Invalid or missing transaction information. `401 Unauthorized` – User is not authenticated. `403 Forbidden` or `404 Not Found` – Account does not belong to the user or cannot be accessed.

Contract field	Team entry
Authentication and authorization	User must be authenticated. Backend verifies that the selected account belongs to the signed-in user before saving the transaction.
Compatibility or versioning	Uses JSON for requests and responses. API changes should be coordinated between frontend and backend to avoid breaking existing functionality.

13 Security and privacy

13.1 Trust boundaries and threats

What to write: *Mark trust boundaries on the context or deployment diagram. Identify likely misuse and failure cases without listing generic security terms.*

Threat or misuse case	Affected asset	Architecture control	Verification evidence	Owner
User requests another user's budget by changing a record ID.	Private budget data	Backend checks the authenticated user's identity and restricts access to their own records.	Negative authorization integration test using two user accounts.	Joey Contreras
Malicious input is submitted through a form and reaches a database operation.	Database integrity and financial records	Validate incoming data and use parameterized database operations through Spring Data JPA.	Input validation tests and review of repository queries.	Joey Contreras
An attacker attempts to access financial information without signing in.	User accounts and financial records	Use Spring Security to protect API endpoints requiring authentication.	Test protected endpoints without valid authentication.	Joey Contreras
An attacker submits incorrect or extreme transaction amounts.	Account balances and transaction records	Validate transaction amounts, formats, and associated account information before saving.	Test negative, positive, zero, and invalid transaction amounts.	Joey Contreras
A user attempts to associate a transaction with another user's account.	Account and transaction data	Verify that the selected account belongs to the authenticated user before creating the transaction.	Attempt to create a transaction using another user's account ID.	Joey Contreras

Threat or misuse case	Affected asset	Architecture control	Verification evidence	Owner
Database credentials are exposed through the shared GitHub repository.	PostgreSQL database and stored financial information	Store production credentials in protected environment variables or secrets instead of committing them to source control.	Review compose.yaml, repository history, and deployment configuration.	Jonathan Ochoa and Joey Contreras
Database becomes unavailable while a user is saving financial information.	Financial records and application availability	Backend handles database connection failures and returns appropriate error responses.	Stop the database container and test backend behavior.	Joey Contreras
A user submits repeated or conflicting updates to an account or transaction.	Account balances and transaction accuracy	Use appropriate database transactions and validation to protect financial record updates.	Test simultaneous updates and verify stored results.	Joey Contreras
Sensitive information is transmitted over an unprotected network connection.	Login credentials and financial information	Use HTTPS/TLS for deployed environments.	Review deployment configuration and verify HTTPS connections.	Jonathan Ochoa and Doga Eski

13.2 Security and privacy decisions

Area	Team decision
Authentication	Use Spring Security to handle authentication and protect backend API endpoints. The specific session or token approach, logout behavior, and expiration settings must follow the backend implementation.
Password handling	Securely hash passwords before storing them in PostgreSQL. Use a Spring Security PasswordEncoder, such as BCrypt. Never store or log plain-text passwords.

Authorization	Restrict users to their own financial records, including accounts, transactions, budgets, loans, and savings goals. Use authenticated user information to verify record ownership. Restrict administrative functions to authorized administrators.
Input and output protection	Use Spring Validation to validate incoming requests. Use Spring Data JPA for database operations and avoid unsafe query construction. Validate financial amounts, dates, categories, and account ownership. Encode user-provided output where appropriate. Establish file validation rules if CSV importing is implemented.
Secrets	Store database credentials and other sensitive configuration in environment variables or a secrets-management solution rather than committing production credentials to GitHub. Restrict access to secrets and rotate credentials when necessary.
Logging	Record application errors, authentication failures, and important system events for troubleshooting. Do not log passwords, authentication tokens, or sensitive financial information.
Financial data	Use fabricated financial and card information for development, testing, and demonstrations. Do not use real banking credentials, actual payment card information, or real banking connections. Users should only access their own financial records.

14 Quality attributes

Quality attribute	Target or scenario	Architecture tactic	Evidence and owner
Performance	Dashboard information should load within 3 seconds under normal demonstration conditions with typical test data.	Use efficient Spring Boot API requests, optimize PostgreSQL queries, and avoid unnecessary frontend requests.	Measure dashboard response time during testing. Joey Contreras and Doga Eski.
Reliability	If the backend or database becomes unavailable, the application should display an appropriate error instead of crashing or displaying incorrect financial information.	Use backend error handling, database transactions where appropriate, input validation, and Docker Compose service dependencies.	Stop and restart containers and verify application recovery. Joey Contreras.
Accessibility	Users should be able to navigate important application features, including login, accounts, transactions, and budgets, using a keyboard and readable interface.	Use semantic React components, accessible form labels, keyboard navigation, and sufficient color contrast.	Perform accessibility and manual keyboard-navigation testing. Doga Eski.
Maintainability	Developers should be able to update a feature, such as budgets or transactions, without unnecessarily changing unrelated application components.	Separate React frontend, Spring Boot backend, REST API interfaces, and database responsibilities. Maintain source code in separate frontend and backend folders.	Review repository organization and perform code reviews. Joey Contreras and Doga Eski.

Quality attribute	Target or scenario	Architecture tactic	Evidence and owner
Testability	Important financial operations should be testable independently, including creating accounts, recording transactions, calculating budgets, and tracking loans.	Use separate backend components, Spring Validation, Spring Data JPA, and fabricated test data.	Perform unit, integration, and manual testing. Joey Contreras.
Observability	When a user encounters an error or a service becomes unavailable, the team should be able to identify which application component failed.	Use application logs, backend error handling, and Docker container logs to identify failures.	Review application and container logs during failure testing. Joey Contreras.

15 Architecture decisions

ADR ID	Decision title	Status	Date	Owner	Related requirements
ADR-001	Use Client-Server and Layered (3-Tier) Architecture	Accepted	11/09/2026	Jonathan Ochoa	System architecture, QA-06
ADR-002	Use React for the frontend	Accepted	11/09/2026	Doga Eski	User interface, UC-AUTH-02, UC-ACC-01

ADR-003	Use Spring Boot for the backend REST API	Accepted	11/09/2026	Joey Contreras	Authentication, financial operations, QA-01
ADR-004	Use PostgreSQL 16 as the application database	Accepted	11/09/2026	Joey Contreras	UC-ACC-01, UC-ACC-02, QA-06
ADR-005	Use Docker Compose to run the application services	Accepted	11/09/2026	Joey Contreras	Deployment, reliability
ADR-006	Use Spring Security for backend authentication and access control	Accepted	11/09/2026	Joey Contreras	UC-AUTH-01, UC-AUTH-02, UC-AUTH-03, UC-AUTH-04, QA-01
ADR-007	Use REST API communication between frontend and backend	Accepted	11/09/2026	Joey Contreras and Doga Eski	Frontend/backend integration, QA-02
ADR-008	Use fabricated financial data instead of real banking integrations	Accepted	11/09/2026	Jonathan Ochoa	Project scope, security and privacy requirements

15.2 Architecture Decision Record template

Field	Content

Decision	A short statement describing the architecture decision the team made. Example: Use React for the frontend application.
Status and date	Identify whether the decision is Proposed, Accepted, or Replaced, followed by the date it was made or documented.
Context	Explain the problem, requirement, constraint, or technical need that led to the decision. Include relevant project requirements, limitations, and goals.
Options considered	List realistic alternatives the team considered. Include options that were rejected and briefly explain why they were not selected.
Choice and reason	Describe the selected option and explain why it was chosen over the alternatives. Include supporting evidence, such as project requirements, existing code, technology compatibility, or prototype results.
Consequences	Describe the benefits, costs, risks, and additional work resulting from the decision. Include any limitations or future considerations.
Owner and reviewers	Identify the person responsible for the decision and the team members who reviewed or approved it.
Related evidence	Reference supporting materials, such as the project charter, requirements document, GitHub repository, source code, tests, prototypes, issues, or related architecture decisions.

16 Traceability and risks

16.1 Architecture traceability

Requirement or quality ID	Architecture element or decision	Verification evidence	Implementation status
UC-AUTH-02 – Create an account	React registration page, Spring Boot backend, authentication component, PostgreSQL database	Demonstrate user registration and verify user information is stored.	Partly implemented / Verify
UC-AUTH-03 – Sign in	React login page, Spring Security, backend authentication	Demonstrate successful and unsuccessful login attempts.	Partly implemented / Verify
UC-AUTH-01 – Reset password	Frontend password recovery page and backend authentication functionality	Demonstrate password recovery process.	Active / Verify
UC-AUTH-04 – Sign out	React frontend and Spring Security authentication	Demonstrate logout and verify protected access afterward.	Active / Verify
UC-ACC-01 – Add a financial account	Account component, REST API, PostgreSQL database	Demonstrate creating, viewing, updating, and deleting accounts.	Active / Verify
UC-ACC-02 – Record a transaction	Transaction component, account component, database	Demonstrate recording income and expenses and checking account balances.	Active / Verify
UC-BUD-01 – Create a monthly budget	Budget component, transaction data, PostgreSQL database	Demonstrate creating a budget and reviewing monthly spending.	Active / Verify
UC-LOAN-01 and UC-LOAN-02 – Add and track loans	Loan component and database	Demonstrate adding loans and displaying balances and due dates.	Active / Verify
UC-GOAL-01 – Create a financial goal	Goal component and database	Demonstrate creating and updating a savings goal.	Active / Verify

Requirement or quality ID	Architecture element or decision	Verification evidence	Implementation status
UC-UC-SPEND-01 – Review spending	Dashboard component, React dashboard, backend API	Demonstrate dashboard information and spending summaries.	Partly implemented / Verify
UC-PROF-01 – Manage profile and settings	React profile pages and backend API	Demonstrate dashboard information and spending summaries.	Active / Verify
QA-01 – Protect user financial information	Spring Security and user-specific data access	Test that one user cannot access another user's financial records.	Verification needed
QA-02 – Dashboard performance	Centralized dashboard API and database queries	Verify dashboard information is retrieved correctly and efficiently.	Verification needed
QA-06 – Persistent financial records	Spring Data JPA, PostgreSQL 16, and CRUD operations	Verify records remain available after application restart.	Verification needed

16.2 Architecture risks and open questions

ID	Risk or question	Why it matters	Validation or response	Owner	Target date	Status
AR-01	Will the Docker Compose environment run all application components reliably during the final demonstration?	Frontend, backend, and database must work together for the application to function.	Run all three containers, verify database connectivity, and test the application from the browser.	Doga Eski and Joey Contreras	Before final demo	Active
AR-02	Are authentication, logout, and password recovery functioning correctly?	Users must be able to securely access their financial information.	Test registration, login, logout, and password recovery.	Joey Contreras	Before final demo	Open
AR-03	Are account balances and transaction calculations accurate?	Incorrect financial calculations could display misleading account information.	Test income, expenses, account balances, and transaction updates with fabricated data.	Joey Contreras	Before final demo	Open

ID	Risk or question	Why it matters	Validation or response	Owner	Target date	Status
AR-04	Does the dashboard correctly display information from all financial features?	Incorrect or missing information could affect the user's understanding of their finances.	Test dashboard integration with accounts, transactions, budgets, loans, and savings goals.	Doga Eski and Joey Contreras	Before final demo	Active
AR-05	Are user profiles and settings fully integrated with the backend?	Users need their profile information and settings to be saved correctly.	Test profile updates and verify that changes persist in the database.	Doga Eski and Joey Contreras	Before final demo	Active
AR-06	Are database credentials protected from exposure?	Credentials committed to GitHub could allow unauthorized database access if reused outside development.	Replace hard-coded development credentials with protected environment variables for deployment.	Joey Contreras	Before final submission	Open
AR-07	Will the team have enough time to complete integration testing and fix remaining issues?	Incomplete features or bugs could affect the final demonstration.	Conduct a full application walkthrough and document unresolved issues.	Jonathan Ochoa	Before final demo	Active
AR-08	Will optional AI and CSV functionality be completed within the remaining project schedule?	Additional features could take time away from completing required functionality.	Prioritize required features and only include optional functionality if time permits.	Jonathan Ochoa and whole team	Before final submission	Open

17 Architecture evolution

The documentation lead reviews this document after almost every sprint. The backend and frontend leads both update their progress on the project weekly on our teams chat. Here are where all of the different updates and conversations are communicated. The team records significant decisions on all related documents after discussing after class or over teams. One of the main discussions that required changes was the AI implementation. We have had many updates on just how much AI would realistically be successfully implemented before our submission deadline.

17.1 Diagram rules

[ ] Give each diagram a title, version or date, purpose, and legend when needed.

[ ] Label arrows with the interaction or protocol, such as HTTPS JSON request or SQL query.

[ ] Use the same component names in diagrams, source folders, interfaces, and tables.

[ ] Use one readable diagram for one main question instead of one giant diagram.

[ ] Mark proposed and implemented elements clearly and update their status each sprint.

[ ] Do not use screenshots of code as architecture diagrams.

18 Review and submission

18.1 Review decisions

Reviewer	Role	Decision	Conditions or comments	Date
[Name]	Project sponsor or mentor	[Approve / Revise]	[Comments]	[Date]
Shah, Syed Jawad Hussain	Instructor	[Approve / Revise]	[Comments]	[Date]
Lupe Campos	Architecture lead	Team acceptance	[Comments]	11/9/2026

18.2 Final submission checklist

[ ] Architecture drivers come from real use cases, nonfunctional requirements, constraints, and risks.

[ ] The document includes context, logical, runtime, development, deployment, and use case views as needed.

[ ] Diagrams use readable labels and names that agree with the implementation.

[ ] The data model protects ownership boundaries and prohibits real financial or card data.

[ ] Interfaces describe authentication, authorization, validation, success, and error behavior.

[ ] Security, secrets, logging, accessibility, reliability, performance, maintainability, and testability are addressed.

[ ] Major decisions include alternatives, rationale, consequences, owner, and date.

[ ] Architecture claims link to requirements and verification evidence.

[ ] Proposed and implemented elements are clearly distinguished.

[ ] All bracketed prompts and unused example rows have been removed.

[ ] The version, date, author list, diagram dates, and review decisions are current.

Recommended filename: TeamName_SystemArchitecture_v1.0.docx